

POWERPOINT 1

Confección de interfaces de usuario

Interfaz de usuario: componente de una aplicación que permite al usuario comunicarse con ella.

Tipos:

- **Textuais CLI** – insercion de ordenes escritas en un intérprete de comandos (terminal, MSDOS, git)
- **Gráficas GUI** – elementos visuales con los que se interactúa
- **Táctiles TUI** – comunicación a través de dispositivo táctil (móviles, TPV)
- **VUI - Voice User Interface** – Siri, Alexa, Google Assistant
- **Gesture-Based Interface** – consolas (Xbox Kinect) y realidad aumentada (Leap Motion, HoloLens)

Componentes

Componente: elemento gráfico atómico que tienen sus propias características y funciones y que se combinan para formar la interfaz.

- **Etiquetas:** Permiten situar un texto en la interfaz. No son interactivos.
- **Campos de texto:** Cuadros de una línea en los que podemos escribir algún dato.
- **Áreas de texto:** Cuadros de varias líneas permite escribir párrafos.
- **Botones:** se pueden pulsar para llevar a cabo alguna acción.
- **Botones de radio:** agrupados para selección de un único elemento entre ellos. El botón marcado se representa mediante un círculo.

-
- **Casillas de verificación:** botones en forma de rectángulo. Se usan para marcar una opción. Cuando está marcada aparece con una marca de verificación dentro de ella. Se pueden seleccionar varias en un conjunto.
 - **Imágenes:** se usan para añadir información gráfica a la interfaz.
 - **Password:** cuadro de texto en el que los caracteres aparecen ocultos.
 - **Listas:** conjunto de datos que se presentan en un cuadro entre los que es posible elegir uno o varios.
 - **Listas desplegables:** combinación de cuadro de texto y lista, permiten escribir un dato o seleccionarlo de la lista que aparece oculta y puede desplegarse.

Bibliotecas de componentes.

Los componentes de una interfaz gráfica suelen agruparse en bibliotecas, que permiten usar clases ya definidas o crear nuevas. La elección de la biblioteca depende principalmente del lenguaje de programación y del entorno de desarrollo utilizado:

- **JAVA Foundation Classes (JFC):** Incluyen las bibliotecas necesarias para crear interfaces gráficas en aplicaciones y applets de Java.
- **AWT:** En desuso. Primera biblioteca de Java para interfaces gráficas. Es común a todas las plataformas, pero cada una tiene sus propios componentes escritos en código nativo.
- **Swing:** Posterior, multiplataforma porque no usa código nativo. Muchos de sus componentes derivan de AWT. Estándar actual para interfaces gráficas en Java
- **JavaFX:** Más moderna para crear interfaces gráficas ricas. Sustituye progresivamente a Swing. Permite trabajar con gráficos 2D y 3D, animaciones, efectos, controles multimedia, hojas de estilo (CSS) y un lenguaje declarativo propio (FXML). Orientado a aplicaciones de escritorio modernas

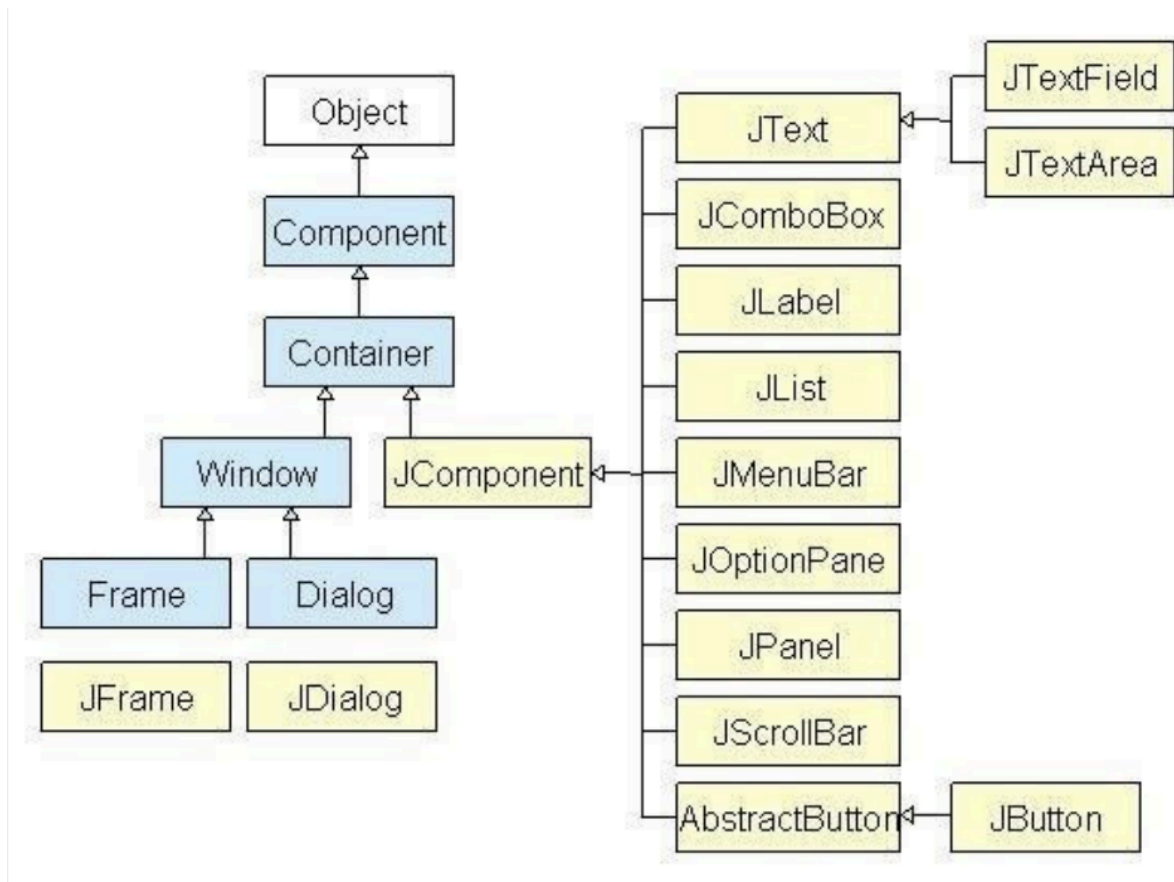
Herramientas para diseñar la interfaz. ¿Wireframe?

Para Swing – WindowBuilder

- Arrastrar y soltar. Genera el código Java de forma automática.

Para JavaFX – JavaFX Scene Builder

- drag & drop
- Guardas el diseño en un archivo FXML y ese archivo se importa en tu proyecto Eclipse y lo vinculas al código controlador.



Colectores

Formulario

- Ventana con botones, barra de título y bordes
- Base para crear aplicaciones de escritorio
- Contiene controles y componentes (Botones, cajas de texto, menú, rejillas de datos)

Contenedores

Un formulario usa contenedores para organizar componentes. Permiten incluir otros componentes y más contenedores y forman una estructura jerárquica.

Contenedor de nivel superior: Contenedor especial que incluye un panel de contenido (content pane). Permite añadir componentes y distribuirlos fácilmente.

Tipos principales:

- **JFrame** (ventana): Ventana con título, botones y bordes
- **JDialog** (diálogo): Ventanas para solicitar información al usuario. Pueden ser modales (bloquean otras ventanas) o no modales
- **JApplet** (applet): Ejecuta aplicaciones Java en una página web. Uso común en aplicaciones de escritorio

Colectores secundarios:

- **Panel de opciones (JOptionPane):** Genera ventanas con botones para responder preguntas con opciones tipo: sí-no, aceptar-cancelar, aceptar, etc.
- **Selector de archivos (JFileChooser)**
- **Selector de colores (JColorChooser):** Permite seleccionar entre un conjunto de colores y devuelve la selección usando el código adecuado.

Otros tipos de colectores:

- **Paneles (JPanel):** Contenedor intermedio, su función principal es colocar controles
- **Barra de menús (JMenu):** Permite la creación de menús complejos de opciones.
- **Barra de herramientas (JToolBar):** iconos de acceso rápido a opciones
- **Pestañas (JTabbedPane):** permite distribuir elementos en pestañas o fichas.
- **Paneles deslizables (JScrollPane):** permite desplazar automáticamente su contenido.
- **Ventanas internas (JInternalFrame):** Ventanas “hijas” que no pueden salir de los límites de la ventana “padre” donde se crearon. Usadas en aplicaciones con varios documentos abiertos simultáneamente.

-
- Paneles divididos (JSplitPane): Permite visualizar dos componentes, uno a cada lado, asignando espacio dinámicamente a cada uno.

Componentes de la interfaz

Componentes: Elementos que se anidan en los contenedores para formar aplicaciones. Se identifican por su clase (función) y su nombre (id único), y poseen propiedades que se pueden modificar: texto mostrado, color, tamaño, etc.

Se usan para solicitar información y para mostrarla.

Disposición de componentes: Layout

Define la posición y orden en el formulario, e interactúan con el usuario por eventos y listeners.

Tipos comunes:

- Bordes (Norte, Sur, Este, Oeste)
- Rejilla (GridLayout)
- Fluido (FlowLayout)

Layouts disponibles y uso en WindowBuilder

Layout	Descripción
BorderLayout	Componentes en Norte, Sur, Este, Oeste
GridLayout	Rejilla con filas y columnas
GridBagLayout	Rejilla flexible, componentes pueden ocupar varias celdas
FlowLayout	Componentes uno tras otro, saltando fila si no hay espacio
GroupLayout	Layout avanzado, usado por defecto en WindowBuilder

POWERPOINT 2

Contenedor:

Para crear una interfaz necesitamos al menos un contenedor.

- **Panel:** contenedor único puro que no es una ventana en sí misma. Su único propósito es organizar los componentes en una ventana.
- **Marco:** Ventana en pleno funcionamiento con su título e iconos.
- **Diálogo:** se puede considerar una ventana emergente que aparece cuando se debe mostrar un mensaje

Contenedores de alto nivel:

- JFrame: habitualmente es el frame principal
- JDialog: ventana de interacción con el usuario

Contenedores intermedios:

- JPanel: contiene otras estructuras
- JScrollPane: incluye barras de desplazamiento

Tipos de layouts:

AbsoluteLayout o NullLayout

Los componentes no se recolocan ni redimensionan automáticamente. No recomendable aunque la ventana no sea redimensionable (el aspecto puede variar entre sistemas)

Problemas de no usar layout:

- Los componentes no se adaptan al redimensionar.
- Interfaz inconsistente entre plataformas.
- Difícil de mantener y modificar.
- Riesgo de solapamiento con textos o fuentes diferentes.

BorderLayout

Divide el contenedor en 5 zonas, pero no es obligatorio usar todas ellas. Permite aplicar otros layout managers dentro de cada panel

Se utiliza con frecuencia para diseñar aplicaciones que tienen zonas fijas (norte, sur, este y/o oeste), donde normalmente se colocan los controles con las acciones principales de la aplicación. La zona central es donde se desarrolla realmente la funcionalidad principal del programa, y es la que aumenta o reduce su tamaño cuando se modifica el tamaño de la ventana o formulario.

FlowLayout

Coloca los componentes en una sola fila, ajustando su tamaño según la propiedad `preferredSize` de cada uno. Si un componente no cabe en la fila actual, se crea una nueva fila y se coloca allí. Pueden estar alineados izquierda/centro/derecha

BoxLayout

Los componentes se colocan uno tras otro si el `BoxLayout` se define como horizontal, o se apilan uno encima de otro si se define como vertical.

GridLayout

Consiste en una rejilla (grid) formada por filas y columnas, donde cada celda tiene el mismo tamaño y cada componente ocupa una celda de esa cuadrícula

GroupLayout

Se reparten los objetos en función unos de otros.

GridBagLayout

`GridBagLayout` es una versión ampliada de `GridLayout`.

Permite crear distribuciones de componentes más complejas. Es, con diferencia, el gestor más completo y flexible. Sin embargo, esta mayor funcionalidad implica una mayor complejidad de uso.

Limitaciones y consideraciones

- Complejidad: Configurar cada componente puede ser tedioso y propenso a errores, sobre todo en formularios grandes.
- VisualBuilder / Eclipse: Aunque VisualBuilder permite diseñar interfaces gráficas, GridBagLayout requiere ajustar manualmente muchas propiedades (gridx, gridy, weightx, weighty), haciéndolo más difícil de manejar que layouts más simples como FlowLayout o BorderLayout.
- No siempre necesario: Para interfaces sencillas o medianas, layouts más simples son más rápidos de configurar y mantener.
- Rendimiento y legibilidad: Interfaces muy complejas con GridBagLayout pueden ser difíciles de mantener y complicar la legibilidad del código, especialmente en proyectos colaborativos. Conclusión GridBagLayout es ideal para diseños complejos y precisos, pero no conviene usarlo para todas las interfaces, sobre todo si se busca rapidez, simplicidad y facilidad de mantenimiento.

2. Máscaras - Look & feel

Permiten cambiar el aspecto visual de las aplicaciones sin modificar su funcionamiento. Un Look & Feel está formado por clases que definen el estilo visual de cada componente de la interfaz (botones, menús, cuadros de texto, etc.). Cuando aplicamos un Look & Feel a una aplicación, le indicamos a Swing que utilice las definiciones gráficas de ese estilo para dibujar los componentes.

Java incluye varios estilos preinstalados que se pueden activar desde el código o el entorno de desarrollo (Metal, Nimbus, Motif, Windows, Windows cLassic)

POWERPOINT 3

3. Menús

Menú: es una representación gráfica que muestra una serie de opciones que una aplicación ofrece. Al seleccionar una opción, se ejecuta una acción relacionada.

En Java existen dos tipos de menús:

- **Menús desplegables:** Son los más comunes. Consisten en opciones organizadas en forma de árbol, que cuelgan de una barra de menú. Al hacer clic en una opción principal, se despliegan los submenús correspondientes.
- **Menús contextuales:** No son visibles inicialmente. Aparecen al realizar una acción específica, (clic derecho), mostrando opciones relacionadas con ese elemento.